



FIREBRAND

LYCEUM EDUCATION GROUP

Data Structures and Their Application

Java Pathway | Session 4

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Data Structures and Their Application

Lesson Objectives

Apply

- Apply appropriate data structures to store and organise data

Analyse

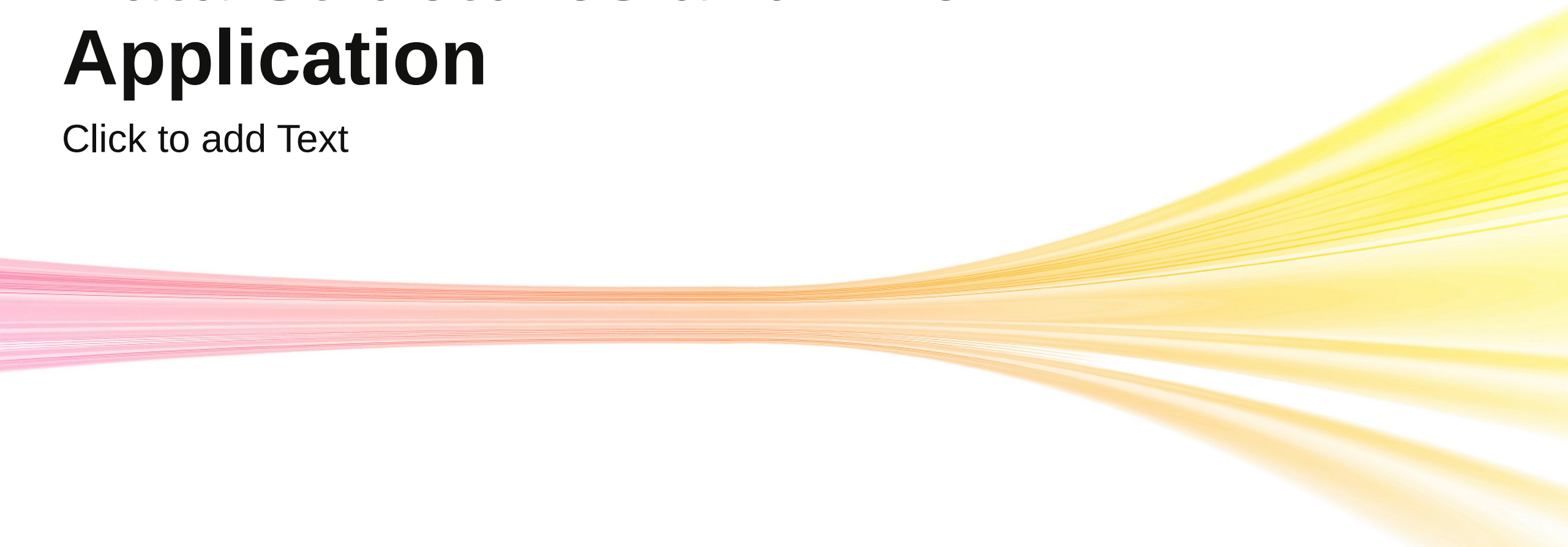
- Analyse differences between arrays, ArrayList, and HashMap

Evaluate

- Evaluate structure suitability based on problem context

Data Structures and Their Application

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient of pink, orange, and yellow, creating a sense of motion and energy.

Introduction to Data Structures

Core Concepts

- **ArrayList** hold resizable, ordered items:

// ArrayList: Ordered and resizable collection

```
ArrayList<String> shoppingList = new ArrayList<>(List.of("Apple", "Banana", "Cherry"));
```

- **Arrays** protect fixed-size groupings:

// Array: Fixed-size sequence

```
double[ ] coordinates = {54.97, -1.61};
```

- **HashMaps** power lightning-fast lookups:

// HashMap: Key-value map pairs

```
HashMap<String, Object> userProfile = Map.of("id", 101, "role", "Admin");
```

Structural Differences

Feature	ArrayList (< >)	Arrays ([])	HashMap ({ })
Ordering	Ordered sequence	Ordered sequence (fixed positions)	No guaranteed order
Mutability	Mutable (Resizable)	Fixed-size (locked)	Mutable keys/values
Access Mode	Integer index (.get(0))	Integer index ([0])	Unique Key (.get("id"))
Duplicates	Allowed entries	Allowed entries	Duplicate keys banned

Syntax Traps

- ArrayList uses **angle brackets** for generics
 < >
- Arrays use **square** brackets
 []
- HashMaps use **.put()** to add key-value pairs
 .put(k, v)

Strengths and Constraints

❑ **ArrayList: Dynamic Growth**

- **Strength:** Easily append elements
- **Constraint:** Slower lookups in large datasets

❑ **Arrays: Fixed-Size Safety**

- **Strength:** Write-protected memory blocks
- **Constraint:** Cannot alter size after initialisation

❑ **HashMap: Key-Value Speed**

- **Strength:** Instant data element lookups
- **Constraint:** Uses significant system memory overhead

Mutability
vs.
Rigid Protection

Real-World Use Cases

❑ **ArrayList: Ordered Data Logs**

- Track incoming sensor streaming data
- Manage linear queue processing lists

❑ **Arrays: Grouped Record Constants**

- Store permanent latitude/longitude coordinate data
- Define fixed RGB colour palette values

❑ **HashMap: High-Speed Lookups**

- Build server status lookup systems
- Manage configuration settings map tables

Mapping Code
to
Business Needs

Right vs. Wrong Collections

Inappropriate Choices:

- **Bad:** Using an ArrayList to look up prices by item name (requires checking every item sequentially)
- **Bad:** Using an array for user shopping carts (crashes when adding items)

Appropriate Replacements

- **Good:** Use a HashMap for rapid keyword lookups
- **Good:** Use an ArrayList for fluid cart additions



FIREBRAND

LYCEUM EDUCATION GROUP

Hands-On Exploration

Practical Exercises: Manipulating Collections

Core Tasks:

- **Modify collections:** Add, remove, and alter elements inside an ArrayList
- **Update HashMaps:** Insert new key-value pairs and change records
- **Access sequences:** Extract elements using numeric indexes and unique keys
- **Loop collections:** Traverse all data structures using iterative structures

Expected Outcome:

- **Working code:** Programs execute smoothly without throwing exceptions
- **Correct types:** Code utilises the correct data structure for each task

Task 1: ArrayList Operations

```
// Create an inventory log ArrayList  
  
ArrayList<String> inventory = new  
ArrayList<>(List.of("Laptop", "Monitor", "Keyboard"));  
  
// TODO: Add "Mouse" to the end of the inventory list  
// TODO: Remove "Monitor" from the collection  
// TODO: Update the first item in the list to "Premium Laptop"  
  
System.out.println("Updated Inventory: " + inventory);
```

Mechanics to Observe:

- **Zero indexing:** Accessing the first item requires index 0
- **Dynamic resizing:** ArrayList size grows automatically after modifications

Task 2: HashMap Lookups

```
// Define a server profile database entry  
HashMap<String, String> server = new  
HashMap<>(Map.of("host", "192.168.1.1", "status", "Online"));  
  
// TODO: Add a new key "port" with the value "8080"  
// TODO: Update the "status" key to "Maintenance"  
// TODO: Access and print the "host" value using its key  
  
System.out.println("Updated Server Data: " + server);
```

Mechanics to Observe:

- **Key collision:** Assigning a value to an existing key overrides the old data
- **Banned indexing:** Attempting `server.get(0)` will fail unless 0 is an explicit key

Task 3: Iteration Experiments

```
// Array of server location coordinates (2D array)
double[][] locations = {{54.97, -1.61}, {51.50, -0.12},
{53.48, -2.24}};

// TODO: Use a for loop to iterate over the locations
array

// TODO: Inside the loop, extract and print the
individual coordinates
```

The Fixed-Size Array Challenge:

- **Experiment:** Try to add a new coordinate (e.g., `locations[3] = new double[]{0.0, 0.0}`)
- **Observation:** Note the exact `ArrayIndexOutOfBoundsException` crash that Java throws to protect the fixed-size structure

CRUD Operations & Summary

Retrieval, Updates, and Collection Iteration

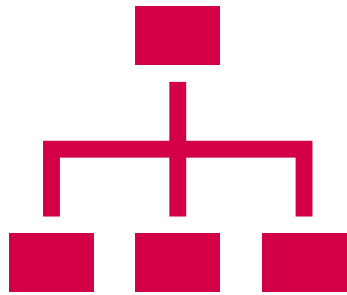
- **Retrieve:** Safely check key presence before extracting quantities
- **Update:** Target nested value coordinates to mutate state
- **Iterate:** Use `.entrySet()` to unpack parent and child fields together

Core Iteration Pattern:

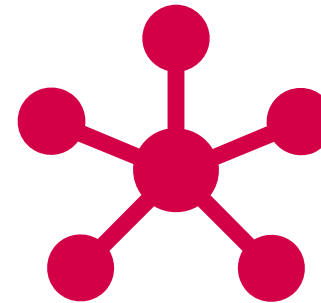
// Unpacking keys and nested containers cleanly

```
for (Map.Entry<String, HashMap<String, Object>> entry : inventory.entrySet()) {  
    System.out.println(entry.getKey() + ": " + entry.getValue().get("qty")); }  
}
```

Stacks & Queues



The Stack Structure
(LIFO)



The Queue Structure
(FIFO)

The Stack Structure (LIFO)

- **Operational Rule:** Last-In, First-Out (LIFO). Elements are added and deleted from the top position
- **Primary Java Methods:** `push(element)` to add data to the top, and `pop()` to remove the top item
- **Real-World System Use:** Browser page-back arrays, expression parser models, or basic program call stacks

The Queue Structure (FIFO)

- **Operational Rule:** First-In, First-Out (FIFO). Elements insert at the tail pointer and exit from the head pointer
- **Primary Java Methods:** `add(element)` to enter the queue back boundary, and `poll()` to dequeue from the front
- **Real-World System Use:** Printer data buffers, messaging systems, or asynchronous thread pools

Linked Lists

Unlike primitive Arrays or dynamic ArrayLists, a **LinkedList** does not use a fixed block of sequential memory spaces. It allocates isolated memory objects called **Nodes**.

Each Node maintains two discrete instance properties:

- **Data Value:** The underlying storage object
- **Next Reference:** A memory pointer linking directly to the adjacent node sequence down the line

```
import java.util.LinkedList;
```

```
LinkedList<String> processChain = new LinkedList<>();
```

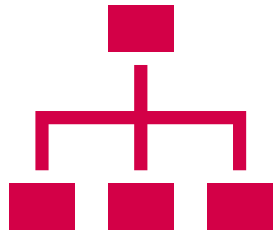
```
processChain.add("Root Node");
```

```
processChain.addFirst("Priority Head Node"); // Updates pointer links instantly
```

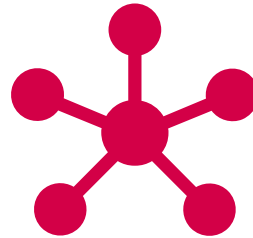
Performance Evaluation

- **Insertions & Deletions:** Extremely fast ($O(1)$ constant time) because updating records only requires rewiring local address pointer properties
- **Data Searching:** Sluggish ($O(n)$ linear time) because the engine must sequentially walk from the Head node down to the target item

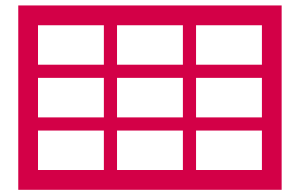
Trees & Graphs



Trees



Graphs



Hash Tables

Hierarchical Systems: Trees

- Organises elements in recursive parent-child pathways starting down from a singular **Root Node**
- **Binary Search Tree (BST):** Enforces a rule where left-child values are always smaller than parent numbers, and right-child values are larger. This strategy reduces search lookups down to $O(\log n)$ logarithmic time efficiency

Interconnected Systems: Graphs

- Represents dynamic networks using collections of value junctions (**Vertices**) connected by relational pathways (**Edges**)
- **Real-World System Use:** Google Maps route mapping, network topology structures, and database link tables

Direct Key Arrays: Hash Tables

- The architectural engine powering the standard Java HashMap
- Passes user keys into a specific parsing formula (**Hash Function**) to generate an integer memory array location index, enabling direct lookup speeds at $O(1)$ constant time efficiency

Lab Activity: Stock-Tracking Prototype

The Mission:

Building a Scalable Nested Inventory Structure

- **Goal:** Build a functional inventory dashboard engine
- **Core Collection:** Implement a nested HashMap pattern
- **Mapping Rule:** Map item string keys to inner property HashMaps

Structural Strategy

```
// Item Name Key -> Maps to inner attribute HashMaps
HashMap<String, HashMap<String, Object>> inventory =
new HashMap<>();

inventory.put("Laptop", new HashMap<>(Map.of(
    "price", 899.99,
    "qty", 15
)));
```

Inventory Management Prototype



ACTIVITY



Objective: Build a small stock-tracking prototype

Tasks / Requirements:

- Store item data using a HashMap of HashMaps (item name -> {price, qty})
- Add functionality to retrieve and update a quantity
- Iterate through the inventory to print a summary

Key Point: Justify why a HashMap was chosen over an ArrayList for this scenario

Extension Activity: Add additional fields such as category or supplier



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation and Comparison Discussion

Analysing Data Structure Trade-Offs

The Three Evaluation Pillars

- **Flexibility vs. Structure:** Balancing the ability to mutate data easily against rigid structure enforcement
- **Read vs. Write Efficiency:** Analysing how choice of collection alters processor speed when looking up or saving data
- **Large-Scale Maintainability:** Evaluating how code adapts when data sets grow from three entries to three million

Expected Outcome

- **Explain Trade-Offs:** Articulate precisely why a chosen structure excels in one area while introducing constraints in another

Flexibility vs. Rigid Structure

Mutable Flexibility (ArrayList & HashMap):

- **The Good:** Dynamic scaling allows items to be added, removed, or updated on the fly
- **The Risk:** Free-form editing leaves records open to accidental overwrite bugs or data type inconsistencies

Structural Protection:

- **The Good:** Write-protected memory blocks guarantee data remains unchanged throughout program lifecycles
- **The Risk:** Absolute immutability requires copying and recreating objects if values ever need to change (**Arrays**)

Read vs. Write Efficiency

Sequential Scans (ArrayList):

- **Write:** Fast - items simply append straight to the end of array memory
- **Read:** Slow - finding a specific item forces a Linear Time check through every single slot

Direct Hashing Maps (HashMap):

- **Write:** Slight memory overhead needed to compute unique internal hash keys
- **Read:** Instant - fetching by direct key identifier scales at lightning-fast Constant Time

Big O Notation (Efficiency)

Measuring Algorithm Scalability

- **Performance metrics:** Evaluates how execution times or resource demands scale as datasets grow
- **Worst-case analysis:** Guarantees structural limits by measuring the absolute maximum boundary of steps
- **Growth tracking:** Shifts focus away from machine processing speeds onto algorithmic efficiency

The Three Core Complexities

- Constant Time: $O(1)$
- Linear Time: $O(n)$
- Quadratic Time: $O(n^2)$

Expected Outcome

- **Optimised Architecture:** Articulate precisely why switching a loop search from an $O(n)$ sequential list scan to an $O(1)$ direct hash key retrieval prevents software degradation at scale

Constant Time: $O(1)$

- **Behaviour:** Performance remains instant and unchanged regardless of dataset scaling
- **Collection Match:** Fetching data via a direct HashMap lookup or accessing an array index slot.

Linear Time: $O(n)$

- **Behaviour:** Execution steps grow in direct, one-to-one proportion to total entries (n)
- **Collection Match:** Traversing an entire ArrayList using a standard for loop to locate an item

Quadratic Time: $O(n^2)$

- **Behaviour:** Execution steps grow exponentially; doubling data quadruples processing overhead
- **Collection Match:** Compiling deeply nested loops or running redundant comparison passes

Cheatsheet

Data Structure	Best Used For...	Biggest Downside	Time Complexity (Big O)
Fixed Array	Storing fixed configuration coordinates (like your Task 3)	Absolutely zero sizing flexibility	Read: $O(1)$ by index / Search: $O(n)$
ArrayList	Creating an ordered log or adding items continuously	Slow searching for non-indexed elements	Write: $O(1)$ append / Search: $O(n)$
HashMap	Dashboards, profile lookups, and fast ID-based queries	High structural memory overhead	Read/Write: $O(1)$ average

Maintainability in Larger Systems

**Schema
Rigidity**

**Team
Readability
Impact**

Stretch and Challenge

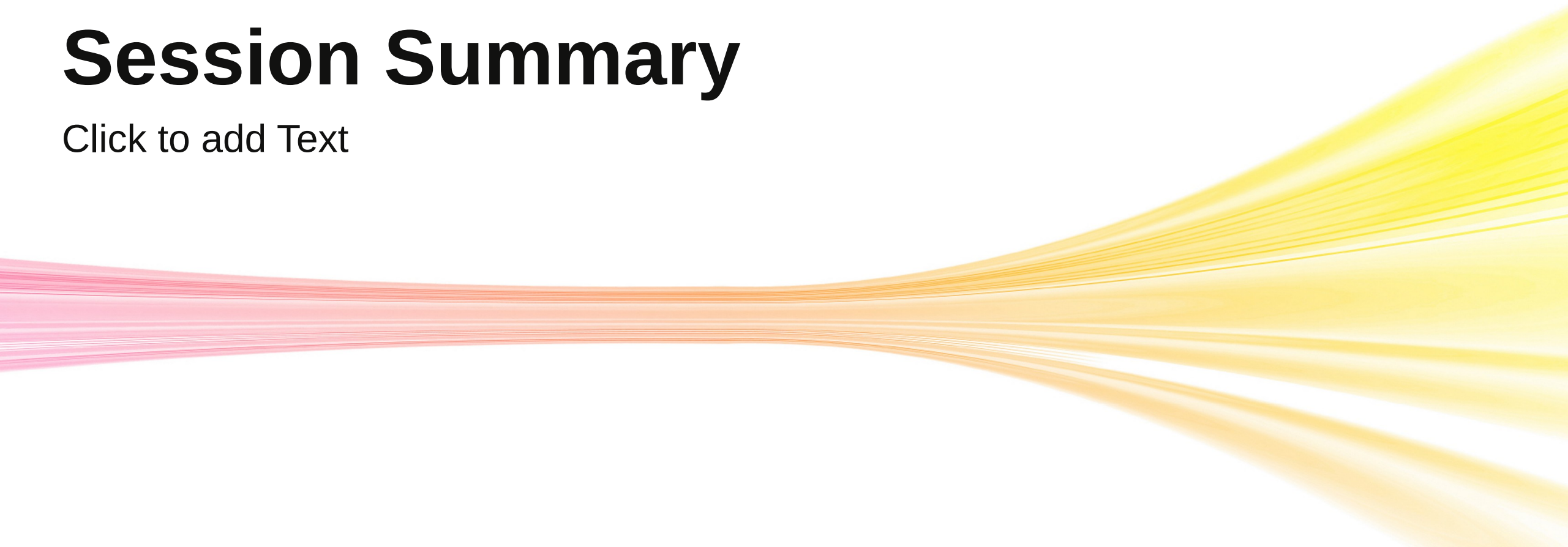
The Mission:

Upgrade your Inventory Management Prototype into production-grade code

- **Task 1:** Explore abstract structures such as stacks and queues
- **Task 2:** Map these concepts to Java implementations using ArrayList or LinkedList
- **Task 3:** Explain how these patterns are used in real-world applications

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Data Structures and Their Application

Lesson Objectives

Apply

- Apply appropriate data structures to store and organise data

Analyse

- Analyse differences between arrays, ArrayList, and HashMap

Evaluate

- Evaluate structure suitability based on problem context

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

